The second matcher, match [a_hash_including(...)], expects our app to return data matching a certain structure; in this case, a one-element array of hashes with certain keys and values. This expression is another use of RSpec's *composable matchers*; here, you're passing the a_hash_including matcher into the match one.

We're deviating a bit from general TDD practice in this snippet. Normally, each example would only have one expectation in it—otherwise, one failure can mask another. Here, we've got two expectations in the same example.

There's a bit of a trade-off to consider here. Any spec that touches the database is going to be slower, particularly in its setup and teardown steps. If we follow "one expectation per example" too rigorously, we're going to be repeating that setup and teardown many times. By judiciously combining a couple of assertions, we're keeping our suite speedy.

Let's see what we're giving up to get this performance boost. Go ahead and run your specs:

```
$ bundle exec rspec spec/integration/app/ledger_spec.rb
« truncated »

Failures:

  1) ExpenseTracker::Ledger#record with a valid expense successfully saves ↩
  the expense in the DB
     Failure/Error: expect(result).to be_success
       expected nil to respond to `success?`
     # ./spec/integration/app/ledger_spec.rb:23:in `block (4 levels) in      ↩
     <module:ExpenseTracker>'

Finished in 0.02211 seconds (files took 0.15418 seconds to load)
1 example, 1 failure

Failed examples:

rspec ./spec/integration/app/ledger_spec.rb:20 #                             ↩
ExpenseTracker::Ledger#record with a valid expense successfully saves the    ↩
expense in the DB

Randomized with seed 27984
```

As you'd expect, the spec failed on the first assertion. We never even see the second assertion, because by default, RSpec aborts the test on the first failure.

It would be nice to record the first failure, but continue to try the second expectation. Good news! The :aggregate_failures tag does just that. Change your example declaration to the following:

06-integration-specs/04/expense_tracker/spec/integration/app/ledger_spec.rb

```
it 'successfully saves the expense in the DB', :aggregate_failures do
```